

Especificação do Projeto Final: Motor de RPG em Texto

Programação III

Objetivo Geral: Desenvolver um motor de jogo de RPG (Role-Playing Game) baseado em turnos executado via terminal, utilizando os pilares da Programação Orientada a Objetos em C++. A modelagem do universo do jogo (nomes de classes, atributos e métodos) fica sob total responsabilidade do desenvolvedor, desde que atenda aos requisitos técnicos obrigatórios descritos abaixo.

Requisitos Técnicos Obrigatórios

Para obter a pontuação máxima, o projeto deve implementar e integrar os 7 conceitos fundamentais de POO:

1. ENCAPSULAMENTO E OCULTAMENTO DE DADOS

- Todos os atributos das classes devem ser protegidos utilizando modificadores de acesso restritos (`private` ou `protected`). Nenhum atributo deve ser exposto publicamente.
- A alteração de qualquer estado ou dado mutável de um objeto deve ser feita obrigatoriamente através de funções de configuração (`setters`), que devem conter validações de consistência lógica (impedindo valores inválidos, como estados negativos ou inconsistentes).

2. CONSTRUTORES E INICIALIZAÇÃO MODERNA

- Cada classe do sistema deve implementar, no mínimo, dois construtores:
 1. Construtor Padrão: Garante a inicialização do objeto com valores seguros de fábrica.
 2. Sobrecarga de Construtor (Parametrizado): Permite a passagem de dados customizados no momento da criação do objeto.
- Atributos que se comportam como constantes do sistema devem ser declarados com o modificador *const* e inicializados obrigatoriamente através da Lista de Inicialização de Membros do construtor.

3. MODIFICADORES DE CONSULTA (CONST-CORRECTNESS)

Todas as funções-membro que realizam apenas a leitura de dados, cálculos matemáticos sem alteração de estado ou exibições de relatórios na tela devem ser marcadas explicitamente com o modificador `const`.

4. SEPARAÇÃO DE INTERFACE E IMPLEMENTAÇÃO

O código do projeto deve ser modularizado. Cada classe implementada deve possuir, obrigatoriamente, a separação em dois arquivos:

- Arquivo de cabeçalho (.h): Contendo estritamente a interface e declaração da classe.
- Arquivo de código-fonte (.cpp): Contendo a definição da lógica das funções-membro.

O programa cliente e a execução do jogo devem residir exclusivamente no arquivo `main.cpp`.

5. HERANÇA E REUTILIZAÇÃO DE CÓDIGO

- O sistema deve possuir uma hierarquia de classes clara, utilizando Herança Pública para criar especializações de entidades a partir de uma classe básica comum (compartilhando e expandindo dados e comportamentos).
- O modificador de acesso `protected` deve ser utilizado de forma estratégica na hierarquia para permitir que as classes derivadas acessem recursos específicos da classe base de maneira controlada.

6. POLIMORFISMO DINÂMICO (FUNÇÕES VIRTUAIS)

- A classe base da hierarquia de combatentes deve declarar pelo menos um método como `virtual`, o qual deve ser devidamente redefinido pelas classes derivadas para expressar comportamentos especializados.
- A classe base deve implementar obrigatoriamente um Destrutor Virtual para assegurar a correta liberação de memória dos objetos.
- A função principal (`main.cpp`) deve demonstrar o uso de Vínculo Dinâmico (`dynamic binding`), processando o turno do jogo por meio de um array ou vetor de ponteiros para a classe básica operando de forma polimórfica.

7. COMPOSIÇÃO E AMIZADE (FRIEND)

- Composição: Pelo menos uma das classes principais deve conter, como membro de dados, um objeto de outra classe independente criada no projeto (estabelecendo a relação de que um objeto "tem um" outro objeto).
- Amizade: O sistema deve conter pelo menos uma relação de Classe Amiga (`friend class`), onde uma classe utilitária de controle externa recebe permissão explícita para

acessar diretamente o escopo privado de outra classe para fins de auditoria ou gerenciamento do jogo.

Diretrizes de Entrega e Compilação

Cada programa de RPG deve ser acompanhado de um relatório que explique a dinâmica do jogo e mostre onde os requisitos técnicos estão sendo aplicados.

O projeto final deve compilar perfeitamente em ambiente de terminal sem apresentar erros ou alertas (warnings).

Data da entrega: **16/07**